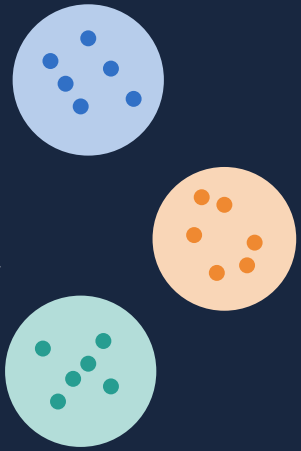
Cluster Analysis

From zero to advanced

A visual, beginner-friendly guide: grouping, distance measures, hierarchical clustering, K-means, DBSCAN, Gaussian mixtures and how to judge your results.



Worked examples • Step-by-step algorithms • Pictures everywhere

Your learning path: from beginner to advanced

LEVEL 1 Beginner

What is a cluster?
Where is it used?
Data types
Distance and similarity

LEVEL 2 Intermediate

Hierarchical clustering
Linkage and dendrograms
K-means step by step
Choosing k (elbow, silhouette)

LEVEL 3 Advanced

Density-based (DBSCAN)
Gaussian mixtures and EM
Validation indices
Choosing a method, Python code

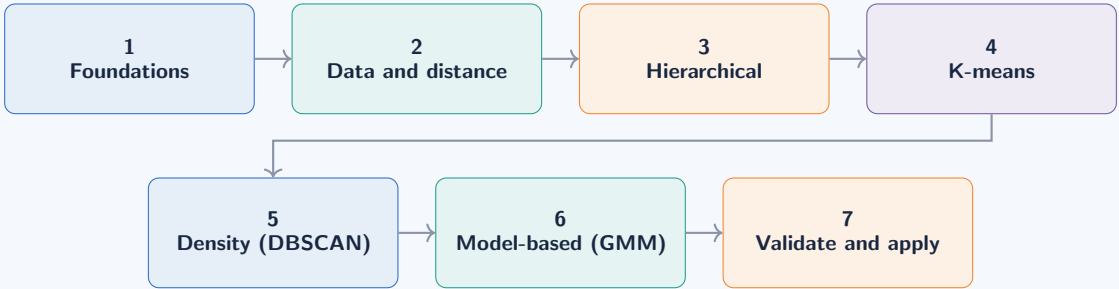


each level builds on the previous one

How to use this deck

Read the **boxes**: teal = beginner tip, orange = worked example, purple = why it matters.

Roadmap: seven parts



Parts 1–2
The vocabulary: what a cluster is and how we measure "closeness".

Parts 3–4
The two classic families: tree-based and centroid-based methods.

Parts 5–7
Modern tools, quality checks, and a practical checklist.

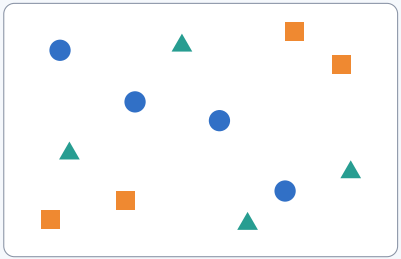
1 Foundations

What is clustering, and why is it useful?

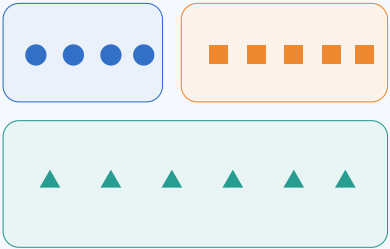


Warm-up: you already know how to cluster

A mixed pile of items



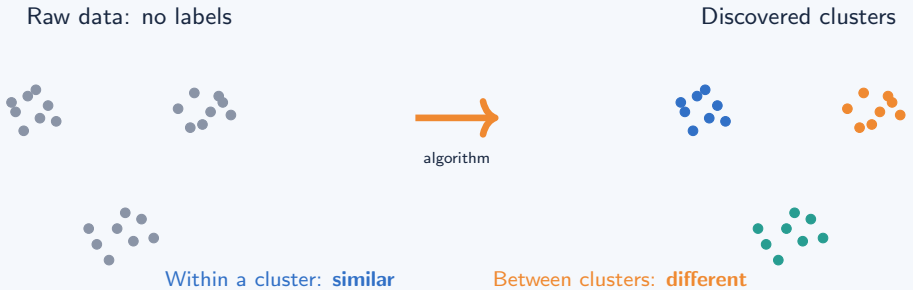
After you group "similar" items



Nobody gave you the group names. You looked at the items and put similar ones together. **That is clustering.**

What is cluster analysis?

Cluster analysis is a family of methods that group objects so that objects in the same group (cluster) are more similar to each other than to objects in other groups.



Where is clustering used?

Marketing

Group customers into segments (bargain hunters, loyal fans) to tailor offers.

Biology and genetics

Find genes that behave alike, or group species into families.

Medicine

Discover patient subtypes that respond differently to treatment.

Text and news

Group articles or reviews by topic without reading each one.

Images

Segment an image into regions, or reduce millions of colors to a palette.

Finance and security

Spot unusual transactions that do not fit any normal group.

Common thread

In every case we have **lots of objects, no ready-made labels**, and we want the data to reveal its own structure.

Supervised vs. unsupervised learning

Supervised (classification)

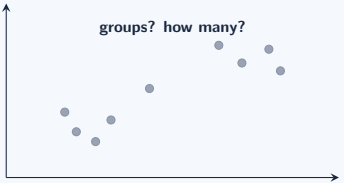
Training examples come with the **right answer** (labels).
Goal: learn to predict the label of a new object.



labels known ⇒ "Which side is the new point on?"

Unsupervised (clustering)

There is **no response variable** telling us the groups.
Goal: let the data suggest the groups.

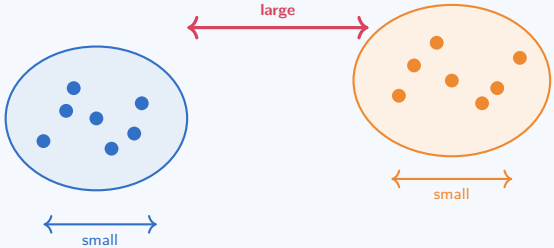


no labels ⇒ "Which points belong together?"

Remember

Clustering is also called **classification analysis**, **numerical taxonomy**, or **segmentation**, depending on the field.

What makes a “good” cluster?



Cohesion
Points inside a cluster are close together (small within-cluster distances).

Separation
Different clusters are far apart (large between-cluster distances).

Good clustering = high cohesion + high separation + patterns that make sense for your problem.

Clusters come in many shapes

Compact blobs



Elongated



Moons



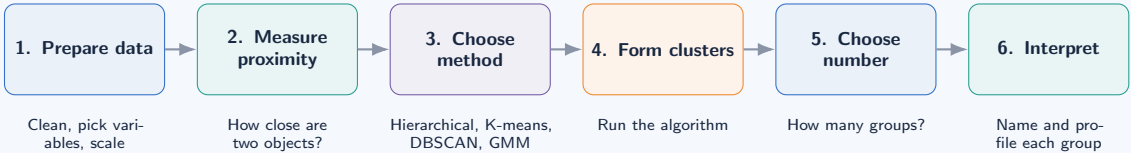
Rings



Big idea for the rest of the course

There is **no single best clustering method**. Each method has an idea of what a cluster looks like: **K-means** likes blobs, **DBSCAN** handles moons and rings, and **Gaussian mixtures** handle stretched ellipses. We will meet them all.

The clustering workflow



Garbage in, garbage out.
Steps 1–2 (data preparation and the distance measure) often matter *more* than the algorithm itself.

It is iterative.
You will typically try several methods or several values of k, compare, and go back.

2

Data and distance

How do we measure “similar”? Everything else depends on this.



The data matrix: objects in rows, variables in columns

Data matrix (n objects $\times$ p variables)

$$X = \begin{pmatrix} x_{11} & x_{12} & \cdots & x_{1p} \\ x_{21} & x_{22} & \cdots & x_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ x_{n1} & x_{n2} & \cdots & x_{np} \end{pmatrix}$$

x_{if} = value of variable f for object i.

Example: 3 customers, 2 variables

Customer	Age	Income (k$)
A	25	40
B	26	80
C	55	41

Each *row* is one customer.
Each *column* is one measured variable.

Idea: each object is a point in p-dimensional space. Customer A is the point $(25, 40)$. "Similar customers" means "nearby points". We will reuse this tiny example on the next slides.

Types of variables (measurement scales)

Type	Meaning	Examples	Typical proximity
Nominal	Categories, no order	colour, city, brand	Matching (same / different)
Binary	Two states, 0/1	owns a car? yes/no	SMC, Jaccard
Ordinal	Categories with order	low $<$ medium $<$ high; star ratings	Ranks mapped to $[0, 1]$
Interval	Equal steps, no true zero	temperature in °C, year	Euclidean, Manhattan
Ratio	Equal steps, true zero	age, income, weight	Euclidean, Manhattan

Beginner tip

Before choosing a distance, **always ask: what kind of variable is this?** Computing an "average colour" of nominal data makes no sense.

Similarity, dissimilarity, distance, proximity

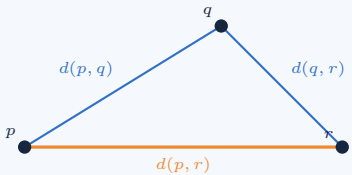
Similarity s
How alike two objects are.
Higher $\Rightarrow$ more alike.
Often in $[0, 1]$.

Dissimilarity d
How different two objects are.
Lower $\Rightarrow$ more alike.
Minimum is often 0.

Proximity
The umbrella word for *either* a similarity or a dissimilarity.

A **distance (metric)** must satisfy, for all objects p, q, r:

1. $d(p, q) \geq 0$ (non-negative)
2. $d(p, q) = 0 \iff p = q$ (identity)
3. $d(p, q) = d(q, p)$ (symmetry)
4. $d(p, r) \leq d(p, q) + d(q, r)$ (triangle inequality)



The direct route is never longer than a detour.

Converting: if $d \in [0, 1]$ then a similarity can be $s = 1 - d$.

Euclidean distance: the straight line

For $p = (p_1, \ldots, p_n)$ and $q = (q_1, \ldots, q_n)$:

$$d(p, q) = \sqrt{\sum_{k=1}^{n} (p_k - q_k)^2}$$

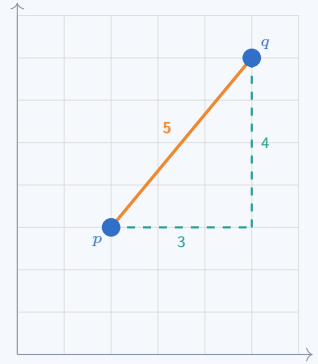
Worked example

$p = (2, 3)$ and $q = (5, 7)$.

$$d(p, q) = \sqrt{(5-2)^2 + (7-3)^2} = \sqrt{9 + 16} = \sqrt{25} = 5$$

Beginner tip

It is just the **Pythagorean theorem**, extended to any number of variables.



Manhattan distance: the city-block walk

Also called **city-block** or L_1 distance:

$$d(p, q) = \sum_{k=1}^{n} |p_k - q_k|$$

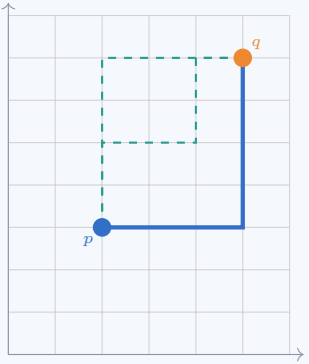
Same points as before

$p = (2, 3)$ and $q = (5, 7)$.

$$d(p, q) = |5 - 2| + |7 - 3| = 3 + 4 = 7$$

Beginner tip

Imagine a taxi in a city grid: it cannot cut through buildings, so it drives along streets.



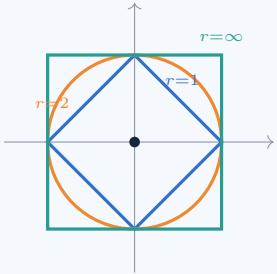
Every staircase path has length 7.

Minkowski distance: one family of distances

$$d_r(p,q) = \Big[\sum_{k=1}^{n} |p_k - q_k|^{\,r}\Big]^{1/r}$$

r	Formula	Name
1	$\sum_k \lvert p_k - q_k\rvert$	Manhattan, L_1
2	$\sqrt{\sum_k (p_k - q_k)^2}$	Euclidean, L_2
∞	$\max_k \lvert p_k - q_k\rvert$	Chebyshev, L_∞

Bigger r gives more weight to the *largest* coordinate difference.



All points at distance 1 from the center: diamond, circle, square.

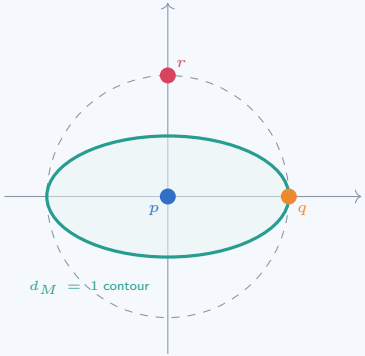
Mahalanobis distance: accounting for spread and correlation

$$d_M(p,q) = \sqrt{(p-q)^\top \, \Sigma^{-1} \, (p-q)}$$

Σ = covariance matrix of the data. If $\Sigma = I$ it becomes Euclidean.

Worked example

Variable 1 has variance 4, variable 2 has variance 1, uncorrelated: $\Sigma = \mathrm{diag}(4, 1)$.
$p = (0, 0)$, $q = (2, 0)$, $r = (0, 2)$.
Euclidean: $d(p,q) = d(p,r) = 2$.
Mahalanobis: $d_M(p,q) = \sqrt{2^2/4} = \mathbf{1}$, $d_M(p,r) = \sqrt{2^2/1} = \mathbf{2}$.



r is "unusual" because variable 2 rarely varies that much.

Use it when

variables are on different scales *and* correlated (e.g. height and weight), and you want distance measured in "standard deviations along the data's own shape".

Why standardization matters

For variable f, the **z-score** is

$$z_{if} = \frac{x_{if} - \bar{x}_f}{s_f}$$

$\bar{x}_f$ = mean, s_f = standard deviation of variable f. After this, every variable has mean 0 and spread 1.

Rule of thumb: **different units or ranges ⇒ standardize first.**

Customers A, B, C

Assume Age: mean 40, sd 10; Income: mean 60, sd 20.

	Raw		z-score	
	Age	Inc.	Age	Inc.
A	25	40	-1.5	-1.0
B	26	80	-1.4	$+1.0$
C	55	41	$+1.5$	-0.95

	Raw	**Standardized**
$d(A, B)$	40.0	**2.00**
$d(A, C)$	**30.0**	3.00

The flip

Raw: A looks closer to C (income 40 vs 41). Standardized: A is closer to B (same age group). **Scale changed the answer.**

Binary variables: Simple Matching vs. Jaccard

object i	1	1	0	0	0	0	0	0	1	0
object j	1	0	0	0	0	0	0	0	1	1

a: both 1 (2) **b**: i=1, j=0 (1) **c**: i=0, j=1 (1) **d**: both 0 (6)

Simple Matching Coefficient

$$\text{SMC} = \frac{a+d}{a+b+c+d} = \frac{2+6}{10} = \mathbf{0.80}$$

Counts shared 1s *and* shared 0s.

Jaccard coefficient

$$J = \frac{a}{a+b+c} = \frac{2}{2+1+1} = \mathbf{0.50}$$

Ignores shared 0s; focuses on shared 1s.

Which one?

If a **0 means "absent" and shared absence is not informative** (two shoppers who both did *not* buy caviar), use **Jaccard**. If 0 and 1 are equally meaningful (male/female), use **SMC**.

Nominal, ordinal and mixed data

Nominal
Categories, no order (red, yellow, blue).
$d = 0$ if same category, $d = 1$ otherwise.

Ordinal
Order matters: low $<$ medium $<$ high.
Replace by ranks r and rescale:

$$z = \frac{r-1}{M-1} \in [0,1]$$

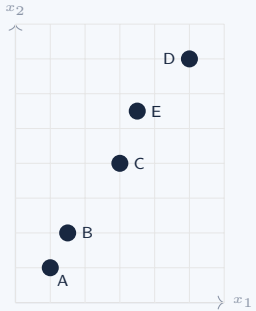
Mixed
A dataset can mix all types. Compute a per-variable distance in $[0, 1]$ and **average** them (Gower).

Worked example: two patients (Gower-style)

Variable	Patient 1	Patient 2	Distance contribution
Gender (nominal)	M	F	1 (different)
Blood pressure (ordinal: low, medium, high)	low	medium	$\lvert 1-2\rvert/(3-1) = 0.5$
Age (interval, range 20–60)	30	50	$\lvert 30-50\rvert/40 = 0.5$

$$d = \frac{1 + 0.5 + 0.5}{3} = \mathbf{0.67}$$

From coordinates to a distance matrix



A(1,1) B(1.5,2) C(3,4)
D(5,7) E(3.5,5.5)

	A	B	C	D	E
A	0	1.12	3.61	7.21	5.15
B	1.12	0	2.50	6.10	4.03
C	3.61	2.50	0	3.61	1.58
D	7.21	6.10	3.61	0	2.12
E	5.15	4.03	1.58	2.12	0

Entry (i, j) = distance between objects i and j; the matrix is **symmetric** with zeros on the diagonal. Darker orange = farther apart. Most algorithms need only this matrix: **distance in, clusters out.**

3

Hierarchical clustering

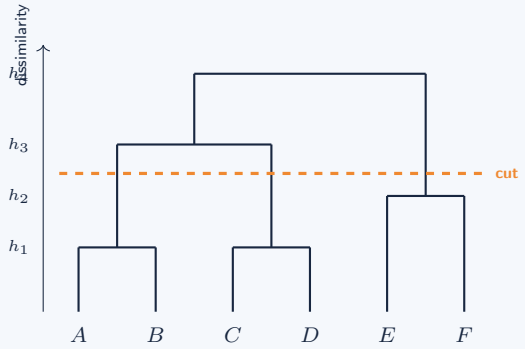
Build a family tree of your data, then decide where to cut.



“‘latex

A dendrogram: the picture behind hierarchical clustering

A **dendrogram** is a tree-like diagram that shows how observations or clusters are merged (or split) at different levels. The vertical axis represents the **dissimilarity** at which the merge occurs.



How to read it

- Leaves are the original observations.
- Lower merges mean greater similarity.
- Higher merges mean greater dissimilarity.
- A horizontal **cut** determines the number of clusters.

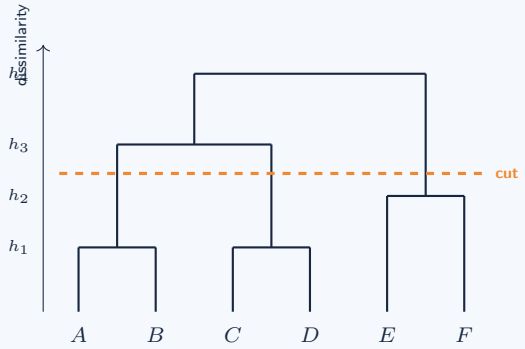
Key idea

Where you cut the tree
determines
how many clusters you get.

Below the cut: **3 clusters** $\longrightarrow$ Move the cut upward $\Rightarrow$ fewer clusters

A dendrogram: the picture behind hierarchical clustering

A **dendrogram** is a tree-like diagram that shows how observations or clusters are merged (or split) at different levels. The vertical axis represents the **dissimilarity** at which the merge occurs.



How to read it

- Leaves are the original observations.
- Lower merges mean greater similarity.
- Higher merges mean greater dissimilarity.
- A horizontal **cut** determines the number of clusters.

Key idea

Where you cut the tree
determines
how many clusters you get.

Below the cut: **3 clusters** $\longrightarrow$ Move the cut upward $\Rightarrow$ fewer clusters

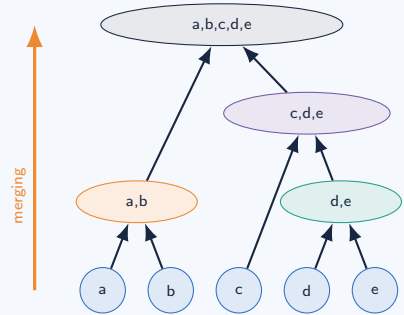
The idea: a tree of nested groups

Agglomerative (bottom-up)
Start: every object is its own cluster.
Repeat: merge the two closest clusters.
End: one big cluster.

Divisive (top-down)
Start: all objects in one cluster.
Repeat: split a cluster in two.
End: every object alone.

Beginner tip

Agglomerative is far more common. We focus on it.



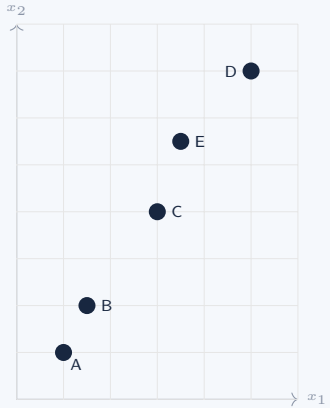
Bottom-up: singletons → pairs → bigger groups → everything.

Agglomerative algorithm in five lines

1. Compute the **distance matrix** between all objects.
2. Put each object in **its own cluster**.
3. Find the two closest clusters and **merge** them. Record the merge height.
4. **Update** the distance matrix using a *linkage rule*.
5. Repeat steps 3–4 until **one cluster** remains.

Decision to make

Step 4 is the only subtle part: **how far apart are two clusters?** That is the linkage rule, coming next.



Our running example: 5 points, $n - 1 = 4$ merges needed.

Linkage: how far apart are two clusters?



Single

nearest pair

$\min d(x, y)$



Complete

farthest pair

$\max d(x, y)$



Average

mean of all pairs

mean $d(x, y)$



Centroid

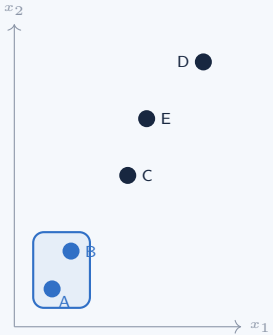
centers

$d(\bar{x}_1, \bar{x}_2)$

Single = optimistic ("are *any* two points close?").
Complete = pessimistic ("are *all* points close?").

Same data + different linkage ⇒ **potentially different clusters**. Ward's method (later) is a fifth popular choice.

Worked example, step 1 of 4 (single linkage)



Distance matrix

	A	B	C	D
B	**1.12**			
C	3.61	2.50		
D	7.21	6.10	3.61	
E	5.15	4.03	1.58	2.12

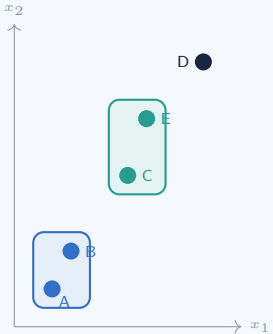
Find the smallest entry: $d(A, B) = 1.12$.
Merge A and B at height **1.12**.
New cluster: {A,B}.

clusters: {A,B}, {C}, {D}, {E}

What happens next

Distances from $\{A, B\}$ to every other object are recomputed: single linkage takes the **minimum** of the two old distances.

Worked example, step 2 of 4 (single linkage)



clusters: {A,B}, {C,E}, {D}

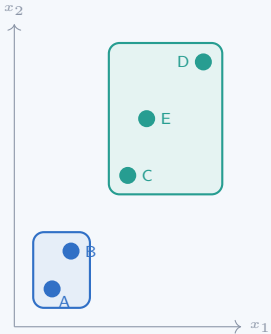
Updated matrix

	AB	C	D
C	2.50		
D	6.10	3.61	
E	4.03	**1.58**	2.12

$d(AB, C) = \min(3.61,\ 2.50) = 2.50$
$d(AB, E) = \min(5.15,\ 4.03) = 4.03$

Smallest entry: $d(C, E) = 1.58$.
Merge C and E at height **1.58**.

Worked example, step 3 of 4 (single linkage)



clusters: {A,B}, {C,D,E}

Updated matrix

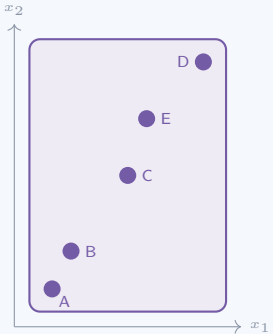
	AB	CE
CE	2.50	
D	6.10	**2.12**

$$d(CE, D) = \min(d_{CD}{=}3.61,\ d_{ED}{=}2.12) = 2.12$$

$$d(AB, CE) = \min(3.61, 5.15, 2.50, 4.03) = 2.50$$

Smallest entry: $d(CE, D) = 2.12$.
Merge D into {C,E} at height **2.12**.

Worked example, step 4 of 4 (single linkage)



one cluster: {A,B,C,D,E}

Final matrix

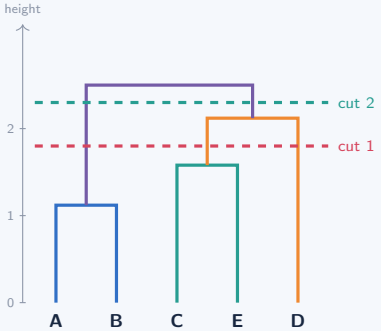
	AB
CDE	**2.50**

$$d(AB, CDE) = \min(2.50,\ 6.10) = 2.50$$

Merge {A,B} and {C,D,E} at height **2.50**. Done: $n - 1 = 4$ merges.

#	Joined	Height
1	A + B	1.12
2	C + E	1.58
3	D + {C,E}	2.12
4	{A,B} + {C,D,E}	2.50

The dendrogram: the merge history as a picture



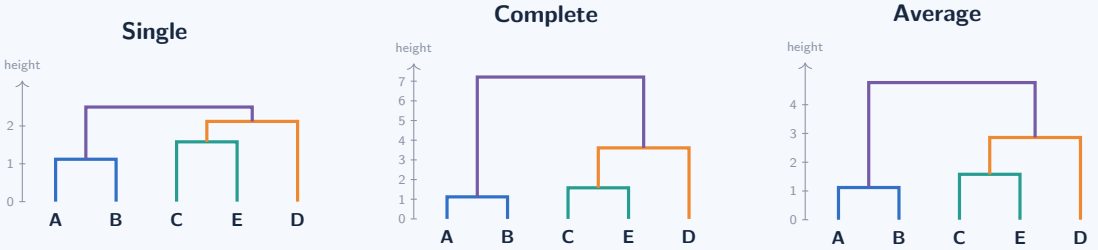
How to read it
Each U-shape is a merge. Its **height** is the distance at which the merge happened. Leaves are objects.

Cutting with a horizontal line gives clusters:
cut 1 at 1.8 $\Rightarrow$ 3 clusters: {A,B}, {C,E}, {D}
cut 2 at 2.3 $\Rightarrow$ 2 clusters: {A,B}, {C,D,E}

Choosing where to cut

Look for a **long vertical gap** with no merges. Higher cut $\Rightarrow$ fewer, larger clusters; lower cut $\Rightarrow$ more, smaller ones.

Same data, different linkage: different merge heights



Merge	AB	CE	D joins CE	root
Single (min)	1.12	1.58	2.12	2.50
Complete (max)	1.12	1.58	3.61	7.21
Average (mean)	1.12	1.58	2.86	4.77

Here the tree shape is the same but **heights differ a lot**; on messier data the shape changes too. Always compare linkages.

Linkage changes cluster shape: the chaining effect



Single linkage follows the bridge point by point and chains both blobs into one long cluster.

Complete / Ward compare whole clusters, so they keep the two compact blobs apart.

Linkage	Tends to produce	Sensitive to
Single	long, straggly chains	noise and bridges
Complete	compact, similar-diameter clusters	outliers
Average	a compromise	moderate
Ward	compact, similar-size clusters	scale of variables

Ward's method: think in terms of variance

Within-cluster sum of squares of a cluster C with centroid $\boldsymbol{\mu}_C$:

$$W(C) = \sum_{\boldsymbol{x} \in C} \|\boldsymbol{x} - \boldsymbol{\mu}_C\|^2$$

Ward merges the pair (C_a, C_b) with the **smallest increase** in total W:

$$\boxed{\Delta = \frac{n_a n_b}{n_a + n_b} \|\boldsymbol{\mu}_a - \boldsymbol{\mu}_b\|^2}$$

Memory cue: Ward → *within-cluster variance*. It tries to keep clusters tight, like K-means does.

Step 1 of our example

For two single points, $n_a = n_b = 1$, so $\Delta = \frac{1}{2} d^2$.

Pair	d^2	Δ
A,B	1.25	**0.63**
C,E	2.50	1.25
D,E	4.50	2.25

Smallest: merge **A,B** first.

Centroid vs Ward: centroid linkage compares cluster centers; Ward asks how much the spread *grows* after merging.

Hierarchical clustering: strengths and limits

Strengths

- No need to fix k in advance
- The dendrogram shows structure at **every** scale
- Deterministic: same input, same output
- Works with any distance matrix

Limits

- Needs the full $n \times n$ matrix: memory $O(n^2)$, time up to $O(n^3)$
- Merges can never be undone (greedy)
- Linkage choice changes the answer
- Hard to read for thousands of objects

Divisive clustering (top-down) runs the story in reverse: start with everything and split repeatedly (e.g. with $k{=}2$ K-means at each step). It is rarer because the number of possible splits is huge.

Rule of thumb

Use hierarchical clustering for **small to medium datasets** (up to a few thousand objects) where you want to *explore* the structure.